PROGRAMA PARA AJUSTE DE CURVAS

Arquitetura de Software e Baseline de Desenvolvimento

Aplicação desktop para curve fitting com modelos arbitrários definidos em LaTeX

Documento	**Architecture Baseline**
Versão	1.5
Status	Baseline revisada - DataSeries, ModelDefinition, ParameterMapper, FitResult e snapshots alinhados às decisões aprovadas
Data	15/10/2026
Escopo	Arquitetura, pacotes, domínio, Model Engine, fitting, resultados, persistência, GUI, testes e conformidade

Este documento é a referência arquitetural normativa do projeto. Alterações estruturais relevantes devem ser confrontadas com esta baseline antes de serem incorporadas.

Sumário

1. Objetivo e escopo
2. Princípios arquiteturais
3. Requisitos funcionais e não funcionais
4. Arquitetura geral
5. Estrutura de pastas e pacotes
6. Modelo de domínio
7. Data Engine
8. Model Engine e interpretação de LaTeX
9. Fitting Engine
10. Modelos padrão
11. Interface gráfica Tkinter
12. Resultados, gráficos e métricas
13. Tratamento de erros e segurança
14. Estratégia de testes
15. Dependências e tecnologias
16. Fluxos principais
17. Roadmap de implementação
18. Regras de conformidade e controle de mudanças
19. Checklist de verificação
20. Arquitetura completa do software
21. Decisões em aberto e extensões futuras
22. Referências técnicas
Anexo A - Critério de aceite da primeira vertical funcional
Anexo B - Registro de decisões arquiteturais

1. Objetivo e escopo

O objetivo é desenvolver uma aplicação desktop em Python, com interface gráfica em Tkinter e arquitetura orientada a objetos, capaz de ajustar conjuntos de dados experimentais a modelos matemáticos conhecidos ou a um modelo arbitrário informado pelo usuário em LaTeX. A V1 executa fitting efetivo com exatamente uma variável independente e uma dependente, mas as estruturas centrais adotam coleções de Variable e matrizes NumPy 2D para permitir evolução futura para múltiplas entradas e saídas sem redesenhar o domínio.

- Importar dados a partir de CSV e Excel e receber dados por copiar/colar.
- Disponibilizar modelos frequentes e modelos customizados explícitos em LaTeX.
- Interpretar a equação simbolicamente, associar variáveis e extrair parâmetros de ajuste.
- Permitir configuração de initial_value, min_value, max_value, vary, name e unit dos parâmetros.
- Executar fitting numérico, retornar parâmetros ajustados, incertezas e métricas.
- Exibir dados experimentais e uma ou mais curvas ajustadas no mesmo gráfico.
- Preservar diferentes FitResult para comparar modelos ou configurações distintas sobre o mesmo Dataset.

Fora do escopo funcional inicial: regressões implícitas F(x,y,...)=0, fitting efetivo com múltiplas variáveis independentes/dependentes, inferência bayesiana, fitting global de múltiplos datasets e processamento distribuído. DataSeries e ModelDefinition permanecem preparados estruturalmente para evolução multidimensional.

2. Princípios arquiteturais

Princípio	Diretriz
Separação GUI / domínio	Tkinter não contém lógica científica. Callbacks delegam a controllers/services.
Representação interna simbólica	LaTeX é formato de entrada/apresentação; SymPy é a representação matemática interna.
Dependência por interfaces	DataLoader e FitEngine possuem contratos abstratos ou interfaces estáveis.
Objetos de domínio estáveis	DataSeries, Dataset, Variable, Parameter, ModelDefinition, FitResult e snapshots representam conceitos independentes da GUI.
Uma única pipeline de modelos	Modelos built-in e customizados convergem para ModelDefinition e usam o mesmo motor de fitting.
Segurança por desenho	Nunca usar eval() sobre entrada matemática do usuário; parsing, whitelist e validação antecedem lambdify.
Testabilidade	Núcleo matemático deve operar sem iniciar Tkinter.
Composição antes de herança artificial	Objetos que compartilham estrutura, mas não semântica de subtipo, devem reutilizar componentes por composição; Dataset e ModelCurve consomem DataSeries.
Snapshot histórico imutável	Resultados de fitting preservam o estado usado na execução sem depender de objetos mutáveis da sessão.
Baseline evolutiva	Mudanças arquiteturais exigem decisão explícita e atualização desta baseline.

3. Requisitos funcionais e não funcionais

3.1 Requisitos funcionais

ID	Requisito
RF-01	Importar CSV com seleção das colunas associadas às variáveis; a V1 seleciona uma independente e uma dependente e converte para matrizes NumPy 2D.
RF-02	Importar XLSX/XLS, selecionar planilha e colunas e converter para a representação matricial canônica.
RF-03	Colar dados tabulares e detectar delimitador/separador decimal.
RF-04	Editar/limpar a tabela de dados.
RF-05	Selecionar modelo built-in.
RF-06	Inserir modelo customizado explícito em LaTeX com símbolos configuráveis para as variáveis.
RF-07	Validar e apresentar a interpretação do modelo antes do fit.
RF-08	Extrair parâmetros automaticamente, excluindo variáveis independentes e constantes reconhecidas.
RF-09	Editar name, unit, initial_value, min_value, max_value e vary de cada Parameter; symbol é a identidade matemática imutável. Parameter não armazena resultado de fitting.
RF-10	Executar fitting e interromper com mensagem controlada em caso de falha.
RF-11	Exibir valores ajustados e incertezas a partir de ParameterResult/FitResult.
RF-12	Calcular e exibir métricas de ajuste.
RF-13	Exibir pontos experimentais e curva(s) ajustada(s) em Matplotlib.
RF-14	Permitir novo fit sem reiniciar a aplicação.
RF-15	Permitir configurar symbol, name e unit das variáveis independente e dependente, usando x/y apenas como defaults.
RF-16	Preservar múltiplos FitResult associados ao mesmo Dataset e permitir selecionar dois ou mais para comparação gráfica.
RF-17	Comparar métricas e resultados de parâmetros entre diferentes fits, inclusive quando modelos distintos reutilizam os mesmos símbolos matemáticos.

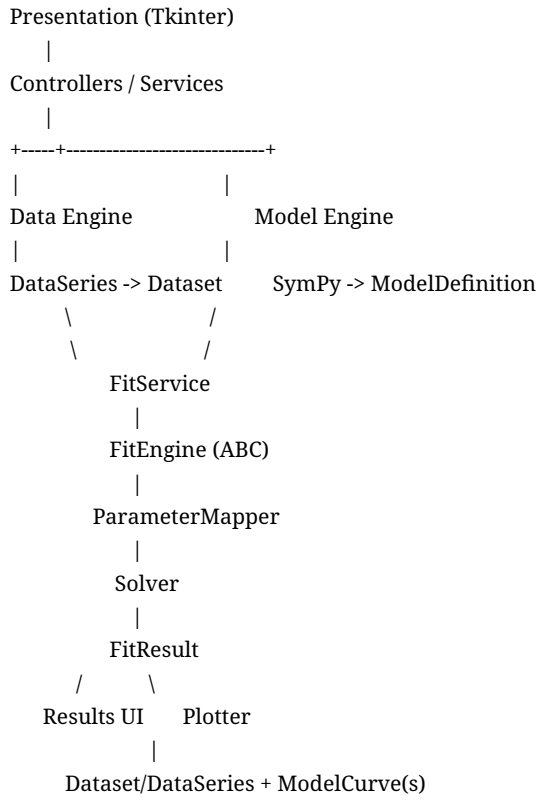
3.2 Requisitos não funcionais

ID	Requisito
RNF-01	Arquitetura orientada a objetos e modular.
RNF-02	GUI responsiva; cálculos demorados não devem congelar permanentemente a interface.
RNF-03	Mensagens de erro compreensíveis e sem traceback bruto para o usuário final.
RNF-04	Entrada matemática não pode executar código Python arbitrário.

RNF-05	Cobertura de testes prioritária para parser, validação, snapshots e fitting.
RNF-06	Reprodutibilidade de resultados dentro das tolerâncias numéricas definidas.
RNF-07	Componentes matemáticos reutilizáveis fora do Tkinter.
RNF-08	Código tipado progressivamente e documentado.
RNF-09	Snapshots e resultados históricos não podem ser alterados por mutações posteriores no ModelDefinition ativo.
RNF-10	Representações serializadas devem ser validadas ao carregar; hashes de integridade detectam alteração de conteúdo, sem substituir assinatura digital.

4. Arquitetura geral

A Presentation Layer captura eventos e exibe estado; controllers traduzem eventos em operações de aplicação; services orquestram objetos de domínio e engines; Data/Model/Fitting/Plotting executam responsabilidades científicas específicas.



Regra central: a GUI nunca manipula diretamente expressões SymPy, não chama SciPy/LMFIT diretamente e não interpreta arquivos. O domínio não depende de GUI, Pandas, Matplotlib ou LMFIT.

5. Estrutura de pastas e pacotes

```
curve_fitting_app/
|-- pyproject.toml
|-- README.md
|-- requirements.txt
|-- src/
|   `-- curvefit/
|       |-- __init__.py
|       |-- main.py
|       |-- domain/
|       |   |-- validation/
|       |   |-- parameter/
|       |   |-- variable/
|       |   |-- data_series/
|       |   |   |-- __init__.py
|       |   |   |-- data_series.py
|       |   |   `-- validator.py
|       |   |-- dataset/
|       |   |-- model_definition/
|       |   |   |-- __init__.py
|       |   |   |-- model_definition.py
|       |   |   `-- validator.py
|       |   |-- snapshots/
|       |   |   |-- parameter_snapshot.py
|       |   |   |-- variable_snapshot.py
|       |   |   `-- model_snapshot.py
|       |   |-- parameter_result.py
|       |   |-- model_curve.py
|       |   |-- fit_result.py
|       |   `-- exceptions.py
|       |-- data/
|       |   |-- data_loader.py
|       |   |-- csv_loader.py
|       |   |-- excel_loader.py
|       |   |-- clipboard_parser.py
|       |   `-- data_validator.py
|       |-- models/
|       |   |-- model_registry.py
|       |   |-- builtin_models.py
|       |   |-- custom_model.py
|       |   |-- latex_parser.py
|       |   |-- symbol_analyzer.py
|       |   `-- numeric_function_builder.py
|       |-- fitting/
|       |   |-- fit_engine.py
|       |   |-- parameter_mapper.py
|       |   |-- lmfit_engine.py
|       |   |-- statistics.py
|       |   `-- initial_guess.py
|       |-- services/
|       |-- gui/
|       `-- plotting/
|           `-- plotter.py
|-- tests/
|   |-- unit/
|   |-- integration/
```

```
|   `-- fixtures/
`-- examples/
```

6. Modelo de domínio

6.1 DataSeries

DataSeries representa a estrutura matemática genérica de pares/matrizes de variáveis independentes e dependentes. Não afirma se os dados são experimentais, simulados ou calculados.

```
DataSeries
- independent_variables: tuple[Variable, ...]
- dependent_variables: tuple[Variable, ...]
- independent_data: ndarray  # shape (n_samples, n_independent_variables)
- dependent_data: ndarray    # shape (n_samples, n_dependent_variables)
```

- independent_data e dependent_data são sempre matrizes 2D float64, com linhas como observações e colunas como variáveis.
- As matrizes possuem o mesmo número de linhas e uma coluna por Variable correspondente.
- A coleção de variáveis é normalizada para tuple; a API pode aceitar Variable isolada ou Sequence[Variable].
- Matrizes armazenadas devem ser protegidas contra mutação externa após validação.
- DataSeriesValidator concentra as validações estruturais antes atribuídas ao DatasetValidator.

6.2 Dataset

Dataset representa dados experimentais e consome DataSeries por composição. Ele acrescenta semântica experimental e metadata, sem duplicar a estrutura X/Y.

```
Dataset
- data_series: DataSeries
- metadata: dict

API delegada:
- independent_variables
- dependent_variables
- independent_data
- dependent_data
- sample_count
```

- A V1 impõe exatamente uma variável independente e uma dependente no Dataset como política de versão.
- DataSeries permanece preparado estruturalmente para múltiplas entradas e saídas.
- Pandas permanece restrito à camada Data e é convertido para ndarray antes do domínio.

6.3 Parameter

```
Parameter
- symbol: str
- name: str | None
- initial_value: float
- min_value: float
- max_value: float
- vary: bool
- unit: str | None
```

- symbol é a identidade matemática imutável local do parâmetro dentro do contexto do modelo.
- name e unit são metadados humanos opcionais.
- Parameter contém somente definição/configuração de entrada; não armazena value, stderr ou qualquer resultado de fitting.
- configure(...) substitui a configuração completa; configure() vazio restaura defaults preservando symbol.
- Alterações inválidas são atômicas e não deixam o objeto parcialmente modificado.

ParameterValidator compõe NumericValidator, TextValidator e SymbolValidator e valida symbol, name, unit, bounds, initial_value e vary. fit_value/stderr deixam de fazer parte do contrato de ParameterValidator.

6.4 Variable

```
Variable
- symbol: str
- name: str | None
- unit: str | None
```

Variable representa uma grandeza observada/modelada. O mesmo tipo é utilizado para entradas e saídas; o papel independente/dependente é definido pelo contexto em DataSeries/ModelDefinition.

6.5 ModelDefinition

```
ModelDefinition
- model_id: UUID
- name: str
- independent_variables: tuple[Variable, ...]
- dependent_variables: tuple[Variable, ...]
- parameters: tuple[Parameter, ...]
- symbolic_expressions: tuple[sympy.Expr, ...]
- numeric_function: callable
```

- model_id é único e imutável por instância de ModelDefinition.
- Uma chamada a copy() cria uma nova instância e, portanto, um novo model_id.
- O construtor pode receber explicitamente model_id apenas em fluxos controlados de restauração/persistência.
- LaTeX não é estado obrigatório de ModelDefinition; é entrada da pipeline do Model Engine.
- symbolic_expressions possuem correspondência posicional 1:1 com dependent_variables.
- numeric_function representa a avaliação vetorizada geral Y = F(X, theta).
- A estrutura matemática do ModelDefinition é somente leitura; Parameter e Variable internos permanecem configuráveis conforme seus próprios contratos.

6.6 ParameterResult

```
ParameterResult
- symbol: str
- value: float
- stderr: float | None
```

ParameterResult contém apenas resultados numéricos de uma execução. Configurações como initial_value, bounds, vary, name e unit não são duplicadas: pertencem ao ModelSnapshot da execução.

6.7 Snapshots imutáveis

VariableSnapshot (frozen)
ParameterSnapshot (frozen)
ModelSnapshot (frozen)
- model_id
- name
- variable snapshots
- parameter snapshots
- symbolic expressions serializáveis/canônicas

- ModelDefinition é operacional e pode ser configurado antes de novos fits; ModelSnapshot é histórico e imutável.
- Snapshots não guardam referências mutáveis aos objetos ativos.
- ModelSnapshot não precisa persistir numeric_function; ela pode ser reconstruída a partir da representação simbólica.
- Dict/JSON são formatos de serialização, não a representação interna principal do snapshot.
- Um SHA-256 pode ser calculado sobre representação canônica dos campos persistentes para verificação de integridade. O hash não substitui assinatura digital.

6.8 ModelCurve

ModelCurve representa uma curva calculada pelo modelo e consome DataSeries. Ela pode usar uma malha independente mais densa que os pontos experimentais, preservando a mesma abstração matricial X/Y usada pelo Plotter.

6.9 FitResult

FitResult
- fit_id: UUID
- model_snapshot: ModelSnapshot
- parameter_results: tuple[ParameterResult, ...]
- residuals: ndarray
- predicted_series: DataSeries
- model_curve: ModelCurve
- covariance: ndarray | None
- metrics: ...
- success: bool
- message: str
- metadata: dict

- fit_id identifica uma execução específica e é distinto de model_id.
- FitResult é um snapshot completo e consistente de uma execução; não possui operação de clear parcial.
- predicted_series contém previsões nos X experimentais para resíduos/métricas.
- model_curve contém a curva suave para apresentação.
- Resultados de parâmetros são associados por symbol dentro do contexto do ModelSnapshot/FitResult; símbolos iguais em modelos diferentes não colidem porque cada FitResult carrega seu próprio model_id via snapshot.

7. Data Engine

O Data Engine converte diferentes origens para DataSeries/Dataset coerentes. Implementações concretas tratam formatos específicos; Pandas pode ser usado para leitura e limpeza tabular, mas não vaza para o domínio.

- CSVLoader: leitura via Pandas, detecção/seleção de delimitador, cabeçalho e colunas.
- ExcelLoader: leitura de workbook e seleção de sheet/colunas.
- ClipboardParser: detecção de tabulação, espaços e vírgulas, com atenção ao separador decimal pt-BR.
- A camada Data converte os dados selecionados para matrizes NumPy 2D float64 e constrói DataSeries; Dataset agrega metadata experimental.
- Ambiguidade decimal deve permitir override explícito de Delimiter e Decimal separator.

8. Model Engine e interpretação de LaTeX

O Model Engine converte a entrada LaTeX em uma ModelDefinition segura, determinística e verificável. O usuário informa apenas a representação LaTeX; SymPy Expr e numeric_function são geradas automaticamente.

8.1 Pipeline normativa

1. Receber a string LaTeX e normalizar espaços/comandos permitidos.
2. Validar a forma geral variável_dependente = expressão e resolver o lado esquerdo.
3. Converter LaTeX para uma ou mais expressões SymPy através de LatexModelParser.
4. Validar funções, operadores e estrutura sintática.
5. Usar ModelSymbolAnalyzer para inspecionar free_symbols, associar variáveis, extrair parâmetros, detectar símbolos desconhecidos e conflitos.
6. Criar Parameter com defaults seguros e símbolos atômicos validados.
7. Gerar uma única numeric_function vetorizada através de NumericFunctionBuilder/lambdify.
8. Validar comportamento numérico e shapes quando aplicável.
9. Construir ModelDefinition com novo model_id.
10. Expor a interpretação de volta à GUI antes de habilitar Fit.

8.2 Modelo matricial geral

A abstração matemática central é Y = F(X, theta), com X de shape (N, n_x) e Y de shape (N, n_y). A representação matricial dos dados não restringe o modelo a relações lineares.

```
X ∈ R^(N x n_x)
Y ∈ R^(N x n_y)
Y_hat = F(X, theta)
```

Quando o modelo é linear nos parâmetros, pode existir a forma especial Y = Phi(X) theta. Isso é uma otimização possível, não um requisito estrutural nem uma classificação por família de equação. Modelos exponenciais, logarítmicos, trigonométricos, potência e combinações continuam na mesma pipeline simbólica.

8.3 Regras de interpretação

- A V1 aceita modelos explícitos com exatamente uma variável independente e uma dependente; contratos internos usam coleções.
- Expressões implícitas, sistemas efetivamente ajustados com múltiplas saídas e ODEs permanecem fora da V1.

- SymbolValidator valida símbolos atômicos; ModelSymbolAnalyzer interpreta o conteúdo das expressões SymPy.
- Funções permitidas inicialmente: exp, log/ln, sin, cos, tan, sqrt e potências; ampliar somente com testes.
- Constantes matemáticas reconhecidas, como pi e e, não são extraídas como parâmetros.
- Não criar grandes blocos condicionais por família de equação; a árvore simbólica dirige a geração numérica.

8.4 Confirmação da interpretação

A GUI mostra a equação interpretada, variáveis e parâmetros antes do fit. O botão Fit permanece desabilitado quando a interpretação é inválida ou pendente. É proibido usar eval() diretamente sobre a entrada.

9. Fitting Engine

FitEngine é uma classe base abstrata (ABC). A implementação inicial é LmfitEngine; engines futuros devem preservar o mesmo contrato.

```
abstract class FitEngine(ABC):
  @abstractmethod
  fit(dataset: Dataset, model: ModelDefinition) -> FitResult
```

Os Parameter já pertencem ao ModelDefinition e não são passados separadamente. O engine cria ParameterMapper no início de cada fitting, capturando a configuração mais recente.

9.1 ParameterMapper

```
ParameterMapper(model.parameters)
- varying_indices
- fixed_indices
- initial_values() -> ndarray shape (n_varying,)
- bounds() -> ndarray shape (2, n_varying)
- expand_values(varying_values) -> ndarray shape (n_parameters,)
```

- ParameterMapper pertence à camada fitting, não ao domínio.
- Ele captura um snapshot operacional da configuração dos parâmetros no momento da criação.
- Parâmetros com vary=False não entram no vetor do solver e usam initial_value como valor fixo ao reconstruir o vetor completo.
- O mapper não executa fitting, não avalia o modelo e não escreve resultados em Parameter.

9.2 Initial guesses

Modelo	Estratégia inicial sugerida
Linear	Regressão linear direta para coeficientes.
Exponencial	Assíntota para C, amplitude inicial para A e inclinação logarítmica aproximada para k.
Gaussiano	Offset pelo baseline, centro no máximo, amplitude pelo pico e sigma pela largura aproximada.
Customizado	Defaults neutros (ex.: 1) e edição manual obrigatoriamente disponível.

9.3 Limites e parâmetros fixos

Parameter aceita initial_value, min_value, max_value e vary. Limites podem ser infinitos para representar ausência de restrição; initial_value deve ser finito e respeitar bounds. Alterar a configuração de Parameter não modifica FitResult anteriores, porque resultados são armazenados em snapshots imutáveis.

10. Modelos padrão

Modelo	Template conceitual	Parâmetros
Linear	y = a x + b	a, b
Polinomial n	y = a_0 + a_1 x + ... + a_n x^n	a_0 ... a_n
Exponencial	y = A e^(B x) + C	A, B, C
Decaimento exponencial	y = A e^(-k x) + C	A, k, C
Logarítmico	y = A ln(x) + B	A, B
Lei de potência	y = A x^n	A, n
Gaussiano	y = A exp(-(x-mu)^2/(2 sigma^2)) + C	A, mu, sigma, C
Logístico	y = L/(1+exp(-k(x-x0))) + C	L, k, x0, C

Modelos padrão são atalhos de entrada. Eles convergem para a mesma pipeline de ModelDefinition usada por modelos customizados; não devem possuir fitting paralelo ou numeric_function manual específica quando a mesma função puder ser gerada simbolicamente.

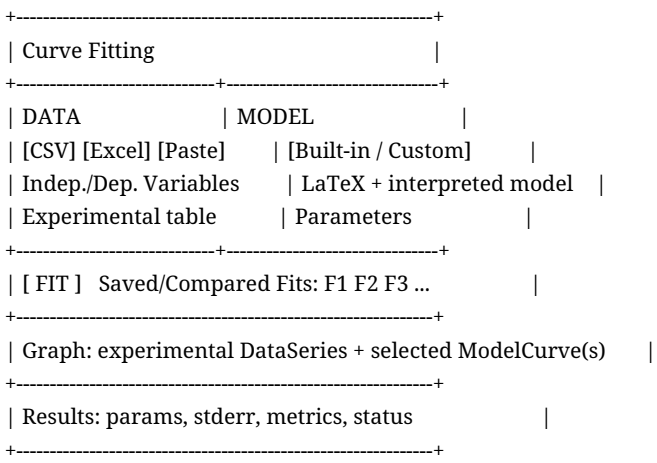
11. Interface gráfica Tkinter

Componente	Responsabilidade
MainWindow	Composição geral, menu, layout e estado global de interação.
DataPanel	Tabela de dados, importar, colar, limpar, selecionar colunas e editar metadados de Variable.
ModelPanel	Escolher built-in ou editar LaTeX; mostrar validação e associação dos símbolos.
ParameterPanel	Exibir symbol e editar name/initial/min/max/vary dos parâmetros descobertos.
ResultsPanel	Exibir status, métricas, ParameterResult e permitir seleção de múltiplos FitResult para comparação.
PlotPanel	Canvas Matplotlib para DataSeries experimental e uma ou mais ModelCurve.
Controllers	Traduzir eventos das views para chamadas aos services.

11.1 Grid de dados

A primeira implementação pode usar ttk.Treeview, com edição de células, seleção de linhas, Ctrl+V, Delete, Import CSV, Import Excel e Clear. A V1 não replica uma planilha completa.

11.2 Layout conceitual



12. Resultados, gráficos e métricas

- O gráfico distingue pontos experimentais das curvas calculadas.
- FitResult armazena predicted_series calculada nos X experimentais para resíduos e métricas.
- ModelCurve é avaliada em uma malha adequada à apresentação e pode possuir mais pontos que o Dataset.
- O Plotter recebe DataSeries/ModelCurve e não executa novo fitting.
- Métricas mínimas: RSS, RMSE e R^2; reduced chi-square apenas quando sua interpretação for válida.
- Parâmetros ajustados são apresentados a partir de ParameterResult, e não de Parameter.
- Diferentes FitResult podem ser sobrepostos no mesmo gráfico e comparados quantitativamente.
- FitResult é imutável como registro histórico; remover um fit da sessão significa remover a referência, não limpar parcialmente o objeto.

13. Tratamento de erros e segurança

Exceção de domínio	Uso
DataImportError	Falha ao ler CSV/Excel ou conteúdo incompatível.
DataValidationError	Dados vazios, shapes incompatíveis, não numéricos ou insuficientes.
ModelParseError	LaTeX não pôde ser convertido para expressão válida.
ModelValidationError	Forma inválida, função/símbolo não permitido ou associação inconsistente.
ParameterError	Initial/min/max/vary inconsistentes.
FitError	Falha numérica, não convergência ou avaliação inválida.
SnapshotValidationError	Snapshot importado, serialização ou hash de integridade inválido.
PlotError	Erro ao renderizar resultados.

- Não exibir traceback bruto ao usuário final.
- Nunca executar código arbitrário vindo do campo de modelo.
- Validar domínio de funções e comunicar valores inválidos.
- Tratar overflow/underflow e NaN/Inf durante avaliação.
- Conservar a aplicação funcional após um fit que falhou.

- Na importação de snapshots/arquivos, validar conteúdo antes de reconstruir objetos de domínio.

14. Estratégia de testes

Camada	Testes mínimos
domain	Construção, invariantes, imutabilidade de snapshots, identidade model_id/fit_id e DataSeries.
data	CSV, XLSX, clipboard, decimal pt-BR, NaN/Inf, colunas inválidas e conversão para DataSeries.
models	LaTeX válido/inválido, SymbolAnalyzer, funções, subscritos, parâmetros, constantes, geração de numeric_function.
fitting	ParameterMapper, parâmetros fixos, bounds, dados sintéticos, erro de convergência, FitEngine abstrato.
statistics	RSS, RMSE, R^2 e casos extremos.
integration	Dataset + ModelDefinition + FitEngine -> FitResult; múltiplos FitResult no mesmo Dataset.
persistence	Round-trip de snapshots, canonicalização e verificação de hash.
gui	Smoke tests de inicialização/controllers sem depender da GUI para validar ciência.

Critério de regressão: toda correção de bug em parsing, fitting, persistência ou invariantes deve adicionar teste que reproduza o caso anterior.

15. Dependências e tecnologias

Tecnologia	Papel	Status arquitetural
Python 3.12+	Linguagem base	Recomendado; validar ambiente final.
Tkinter / ttk	GUI desktop	Obrigatório pelo escopo atual.
NumPy	Matrizes canônicas, vetores do solver, álgebra e avaliação vetorizada	Core científico.
Pandas	Importação/manipulação tabular antes da conversão para NumPy	Somente camada Data.
SymPy	Parsing, representação simbólica, análise e lambdify	Model Engine.
LMFIT	Fitting não linear inicial	Implementação inicial de FitEngine.
SciPy	Base numérica/possível engine futuro	Borda de fitting.
Matplotlib	Plotagem incorporada ao Tkinter	Output visual.
pytest	Testes	Recomendado.
hashlib/json	Canonicalização e SHA-256 de snapshots persistidos	Biblioteca padrão; integridade, não assinatura digital.

16. Fluxos principais

16.1 Importação e preparação de dados

```
User -> DataPanel -> DataController -> DataService
    -> CSVLoader / ExcelLoader / ClipboardParser
    -> DataValidator -> DataSeries -> Dataset -> estado da aplicação
```

16.2 Construção de modelo customizado

User LaTeX -> ModelPanel -> ModelController -> ModelService
-> LatexModelParser -> SymPy Expr
-> ModelSymbolAnalyzer -> Parameter
-> NumericFunctionBuilder -> ModelDefinition(model_id)
-> GUI mostra interpretação + parâmetros

16.3 Fitting

Dataset + ModelDefinition
-> FitController -> FitService -> FitEngine
-> ParameterMapper -> solver -> Statistics
-> ModelSnapshot + ParameterResult + DataSeries/ModelCurve
-> FitResult(fit_id)
-> ResultsPanel + Plotter -> PlotPanel

16.4 Comparação de fits

Dataset + [FitResult F1, F2, ...]
-> ResultsPanel seleciona resultados
-> Plotter recebe experimental DataSeries + ModelCurve(s)
-> comparação gráfica e quantitativa sem recalcular fitting

17. Roadmap de implementação

Fase	Entrega
Fase 1 - Domain	Variable, Parameter, DataSeries, Dataset, ModelDefinition, snapshots, ParameterResult, ModelCurve, FitResult, validadores e exceções.
Fase 2 - Model Engine	LatexModelParser, ModelSymbolAnalyzer, extração de parâmetros, NumericFunctionBuilder/lambdify e ModelRegistry.
Fase 3 - Fitting	FitEngine, ParameterMapper, LmfitEngine, statistics, dados sintéticos e testes.
Fase 4 - Data Engine	CSV, XLSX, clipboard e validação.
Fase 5 - Services	DataService, ModelService e FitService.
Fase 6 - GUI estrutural	MainWindow, panels, widgets e controllers.
Fase 7 - Integração	Fluxo ponta a ponta, múltiplos FitResult, comparação e tratamento de erros.
Fase 8 - Persistência/polimento	Snapshots exportáveis, integridade, UX, responsividade, logs, exportações e empacotamento.

Ordem recomendada: validar DataSeries/Dataset -> LaTeX -> ModelDefinition -> ParameterMapper/FitEngine -> FitResult com testes automatizados; depois acoplar Tkinter.

18. Regras de conformidade e controle de mudanças

11. Antes de criar ou modificar um módulo estrutural, verificar pacote e responsabilidade nesta baseline.
12. Se o código proposto divergir da baseline, informar explicitamente a divergência antes de consolidá-la.
13. A análise deve identificar o que diverge, por que diverge, impacto e recomendação: alterar código ou revisar documentação.

14. Não alterar silenciosamente a arquitetura por conveniência local.
15. Quando uma mudança arquitetural for aprovada, atualizar a versão deste documento e registrar a decisão.
16. Refatorações internas que preservem contratos podem ocorrer sem nova baseline, desde que mantenham testes.

Nível	Exemplo	Ação
Baixa	Nome de método/classe diferente, mesma responsabilidade	Registrar na revisão; adaptar documentação se persistente.
Média	Mudança de dependência, novo serviço, reorganização de pacote	Apresentar antes de consolidar; decidir código x documento.
Alta	GUI executando lógica científica, eliminação de Model Engine, uso de eval, violação dos snapshots	Bloquear como baseline violation até decisão explícita.

19. Checklist de verificação

[ ] A GUI não contém lógica de parsing ou otimização.

[ ] DataSeries usa matrizes NumPy 2D canônicas e não depende de Pandas/Tkinter.

[ ] Dataset consome DataSeries por composição e representa semântica experimental.

[ ] LaTeX é convertido para SymPy antes do fitting e não é necessário como estado do ModelDefinition.

[ ] Não existe eval() de entrada matemática do usuário.

[ ] ModelSymbolAnalyzer distingue variáveis, parâmetros e símbolos inválidos de forma determinística.

[ ] Modelos built-in e customizados resultam em ModelDefinition.

[ ] Cada ModelDefinition possui model_id único e imutável; copy() gera novo model_id.

[ ] Parameter contém somente configuração; resultados ficam em ParameterResult/FitResult.

[ ] ParameterMapper pertence ao fitting e é criado no início de cada execução.

[ ] FitEngine possui contrato abstrato independente da implementação LMFIT.

[ ] FitResult possui fit_id e ModelSnapshot imutável.

[ ] Snapshots não retêm referências mutáveis aos objetos ativos.

[ ] Hash de snapshot é calculado sobre representação canônica quando usado para integridade.

[ ] predicted_series e model_curve são conceitos distintos.

[ ] Plotter usa DataSeries/ModelCurve e não recalcula fitting.

[ ] É possível preservar e comparar múltiplos FitResult para o mesmo Dataset.

[ ] Data Engine trata CSV, Excel e clipboard separadamente.

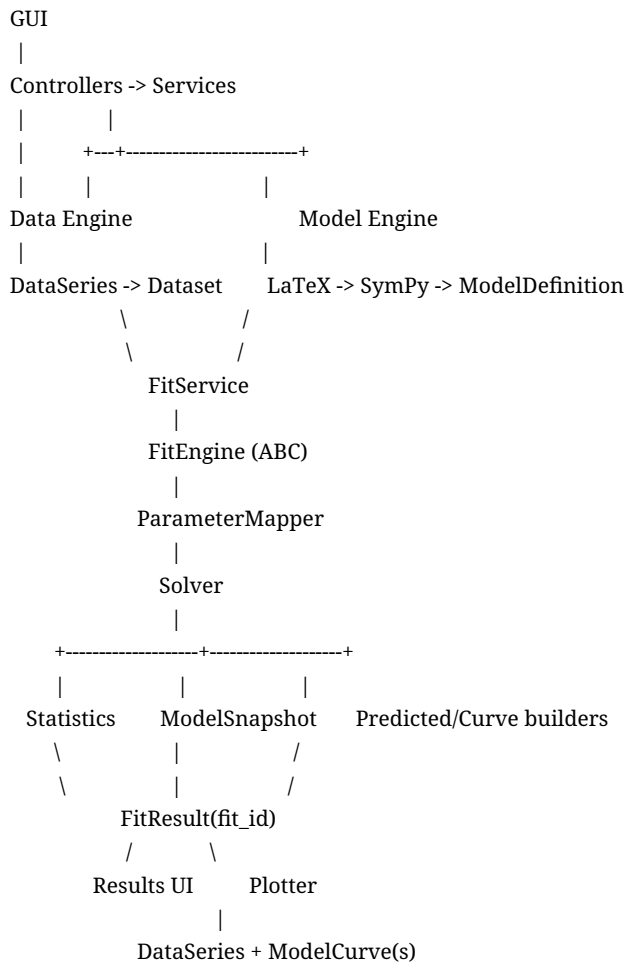
[ ] Parser e fitting possuem testes independentes de Tkinter.

[ ] Dados sintéticos recuperam parâmetros conhecidos dentro de tolerância.

[ ] Toda divergência arquitetural relevante foi explicitamente avaliada.

20. Arquitetura completa do software

O diagrama abaixo consolida as responsabilidades e dependências principais. Setas indicam fluxo/dependência, não necessariamente chamadas síncronas exclusivas.



20.1 Dependências permitidas

- gui -> controllers -> services.
- services -> domain e interfaces de engines.
- data -> domain.DataSeries / Dataset.
- models -> domain.ModelDefinition / Parameter / Variable e SymPy.
- fitting -> domain + biblioteca numérica; ParameterMapper reside em fitting.
- plotting -> DataSeries / Dataset / ModelCurve / FitResult + Matplotlib.
- domain pode usar NumPy/SymPy quando explicitamente previsto pelo contrato, mas não depende de GUI, Pandas, Matplotlib ou LMFIT.

20.2 Dependências proibidas

- domain -> Tkinter/Pandas/Matplotlib/LMFIT.
- ModelDefinition -> fitting.ParameterMapper.
- views -> scipy.optimize ou lmfit diretamente.
- plotter -> executar novo fit.
- CSVLoader/ExcelLoader -> atualizar widgets Tkinter.
- LatexModelParser -> exibir messagebox ou alterar estado da GUI.
- FitResult -> objetos mutáveis vivos da sessão que permitam alterar retroativamente o histórico.

21. Decisões em aberto e extensões futuras

Tema	Decisão atual	Possível evolução
Parser LaTeX	Backend encapsulado com parser SymPy/Lark validado por testes.	Gramática própria ou MathLive/webview se UX exigir.
Fitting	LMFIT como engine inicial atrás de FitEngine ABC.	SciPyFitEngine, robust loss, BayesianFitEngine.
Planilha	ttk.Treeview.	Widget tabular especializado se edição avançada se tornar requisito.
Threading	Fit síncrono nos primeiros testes de núcleo.	Worker thread/queue para datasets/modelos lentos.
Exportação	FitResult + snapshots preparados conceitualmente para serialização; formato final ainda não fechado.	CSV de parâmetros, JSON de análise, PNG/SVG e relatório PDF.
Modelos	V1: uma independente e uma dependente; contratos matriciais e coleções já são gerais.	Múltiplas entradas/saídas, global fit, múltiplos datasets, implícitos e ODEs.
Persistência	Sessão em memória; snapshots imutáveis definidos.	Salvar/carregar projeto e um FitRecord/AnalysisRecord que inclua também snapshot completo do Dataset para reexecução integral.
Integridade	SHA-256 sobre conteúdo canônico de snapshot quando exportado.	Assinatura digital/HMAC se autenticidade contra adulteração se tornar requisito.

22. Referências técnicas

- SymPy - parsing LaTeX, representação simbólica, free_symbols, srepr e lambdify.
- LMFIT - parâmetros, minimização, limites e incertezas.
- SciPy - scipy.optimize, least_squares e curve_fit.
- NumPy - arrays, broadcasting, operações vetorizadas e vetores de solver.
- Pandas / openpyxl - importação/manipulação CSV/XLSX antes da conversão para NumPy.
- Matplotlib - integração com Tkinter via FigureCanvasTkAgg.
- Python abc - classes abstratas e abstractmethod.
- Python dataclasses - snapshots imutáveis com frozen=True.
- Python uuid - model_id e fit_id.
- Python hashlib/json - canonicalização e SHA-256 de integridade.
- Python tkinter / ttk - widgets desktop e event loop.

Anexo A - Critério de aceite da primeira vertical funcional

17. Usuário cola ou importa um dataset com duas colunas.
18. A aplicação constrói DataSeries validado e o agrega a um Dataset.
19. Usuário seleciona um modelo built-in ou informa T = A e^{-k t} + C em LaTeX.
20. O Model Engine gera SymPy, identifica t/T, cria A/k/C como Parameter e constrói ModelDefinition com model_id.
21. O usuário edita name/initial/min/max/vary dos parâmetros.
22. FitService executa FitEngine; o engine cria ParameterMapper sem a view importar LMFIT/SciPy.
23. O solver retorna valores que originam ParameterResult, ModelSnapshot, predicted_series, ModelCurve e FitResult com fit_id.
24. ResultsPanel mostra parâmetros/métricas e PlotPanel exibe dados + curva.
25. O usuário pode alterar/copiar um modelo, executar outro fit e manter ambos os FitResult para comparação.
26. Um conjunto de testes de integração reproduz o mesmo fluxo sem abrir Tkinter.

Anexo B - Registro de decisões arquiteturais

ID	Decisão	Status
ADR-001	Separar GUI e núcleo científico.	Aprovado na baseline v1.1
ADR-002	Usar LaTeX apenas como entrada/apresentação e SymPy como representação interna.	Aprovado na baseline v1.1; reforçado na v1.5
ADR-003	Encapsular parsing em LatexModelParser.	Aprovado na baseline v1.1
ADR-004	Não utilizar eval() em entrada do usuário.	Obrigatório
ADR-005	Adotar FitEngine abstrato e LmfitEngine inicial.	Aprovado na baseline v1.1; contrato atualizado na v1.5
ADR-006	Modelos built-in e customizados convergem para ModelDefinition.	Aprovado na baseline v1.1
ADR-007	Introduzir Variable para grandezas independente/dependente com symbol/name/unit.	Aprovado na baseline v1.1
ADR-008	Separar identidade matemática e descrição humana: Parameter.symbol matemático; name descritivo opcional.	Aprovado na baseline v1.1
ADR-009	Tratar x e y apenas como símbolos padrão, sem transformá-los em Parameter.	Aprovado na baseline v1.1
ADR-010	Contrato antigo de Parameter incluía value/stderr e clear_fit_result().	SUBSTITUÍDO pela ADR-014 na baseline v1.5
ADR-011	Introduzir TextValidator e SymbolValidator reutilizáveis; ParameterValidator compõe validadores genéricos.	Aprovado na baseline v1.3; ajustado na v1.5 para remover fit_value/stderr
ADR-012	Adotar matrizes NumPy 2D canônicas para dados, com coleções de Variable e Pandas restrito à camada Data.	Aprovado na baseline v1.4; generalizado por DataSeries na v1.5
ADR-013	Introduzir DataSeries como estrutura genérica X/Y e compor Dataset e ModelCurve a partir dela.	Aprovado na baseline v1.5
ADR-014	Parameter contém apenas definição/configuração; resultados de fitting ficam em ParameterResult/FitResult.	Aprovado na baseline v1.5
ADR-015	ModelDefinition possui model_id UUID único por instância; copy() gera novo model_id; restauração pode fornecer ID persistido.	Aprovado na baseline v1.5
ADR-016	ModelDefinition armazena symbolic_expressions e uma única numeric_function Y=F(X,theta); LaTeX	Aprovado na baseline v1.5

	permanece na pipeline de entrada.	
ADR-017	ModelSymbolAnalyzer assume análise semântica dos símbolos; ModelDefinitionValidator permanece estrutural.	Aprovado na baseline v1.5
ADR-018	ParameterMapper reside em fitting, é instanciável e criado pelo FitEngine no início de cada ajuste.	Aprovado na baseline v1.5
ADR-019	FitEngine usa ABC/@abstractmethod e contrato fit(dataset, model) -> FitResult.	Aprovado na baseline v1.5
ADR-020	FitResult é snapshot imutável de uma execução, possui fit_id, ModelSnapshot, ParameterResult, predicted_series, residuals, métricas e ModelCurve.	Aprovado na baseline v1.5
ADR-021	ModelSnapshot, ParameterSnapshot e VariableSnapshot são imutáveis; serialização é separada da representação interna.	Aprovado na baseline v1.5
ADR-022	Snapshots podem expor SHA-256 de integridade sobre representação canônica; não equivale a assinatura digital.	Aprovado na baseline v1.5
ADR-023	A V1 deve preservar múltiplos FitResult para comparação gráfica e quantitativa no mesmo Dataset.	Aprovado na baseline v1.5